Project 2 Report

Name of Team Members: Lance Penaflor and Jay McIntosh

Date: October 20, 2026

Experiment

To analyze the performance of multi-threaded computation, we conducted a series of automated runtime experiments alongside manual correctness testing.

For the performance benchmark, we utilized a Bash script to execute the mt-collatz program with the upper bound fixed at $N = 5{,}000{,}000$. We varied the thread count from $T = 1$ to $T = 8$. To account for operating system scheduling jitter, we repeated the execution 10 times for each thread count configuration, both with mutex locks enabled and disable. The elapsed wall clock times were measured down to the nanosecond using clock_gettime(), and the 10 runs were averaged to produce our final runtime results.

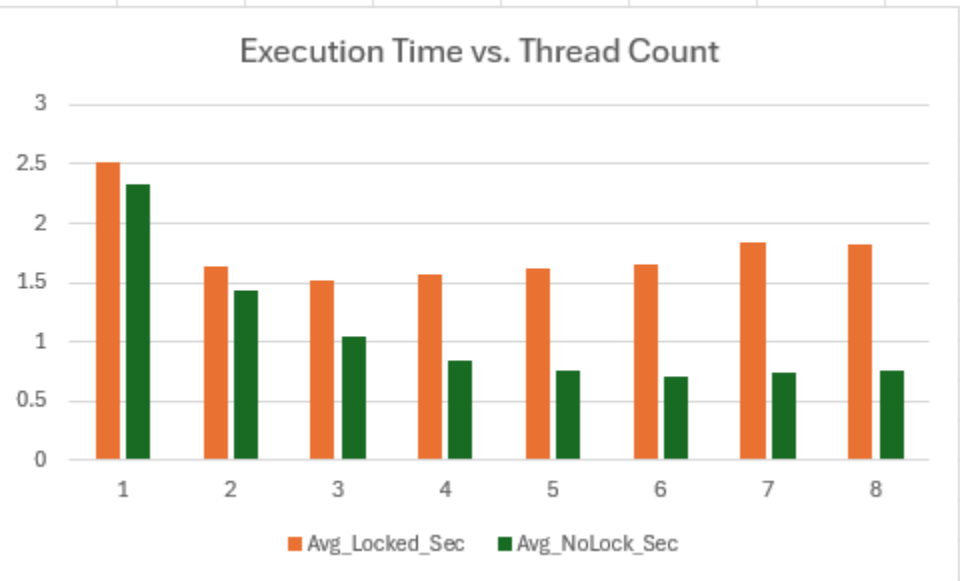
To verify race conditions and program correctness, we executed the specific test cases provided in the assignment outline at $N = 500{,}000$:

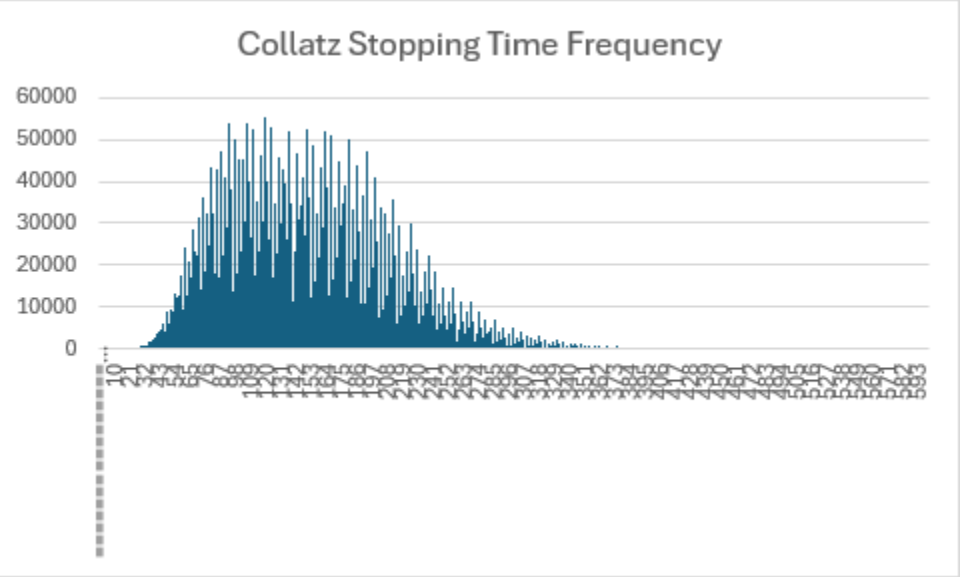
1. We generated a single-threaded baseline without locks: ./mt-collatz 500000 1 -nolock > hist1.txt
2. We generated a multi-threaded run without locks: ./mt-collatz 500000 5 -nolock > hist2.txt
3. We compared the outputs using the diff command.
4. We generated a multi-threaded run with locks: ./mt-collatz 500000 5 > hist3.txt
5. We compared hist1.txt and hist3.txt using the diff command.

Experimental Environment:

All experiments were conducted on the university's department Linux server (cs-ssh.argo.uwf.edu). The machine operates on an Intel(R) Xeon(R) Gold 6338N CPU @ 2.20GHz.

Results





Average Time Measurement Results (N = 5,000,000):

Threads (T)	Avg Runtime Locked (s)	Avg Runtime -nolock (s)
1	2.523339	2.327019
2	1.637258	1.424921
3	1.518114 (Best)	1.039241

Threads (T)	Avg Runtime Locked (s)	Avg Runtime -nolock (s)
4	1.571487	0.841896
5	1.622606	0.755817
6	1.651248	0.705526 (Best)
7	1.837880	0.734469
8	1.817836	0.749128

Discussion

Our analysis of the runtime results revealed a contrast between unsynchronized parallel computation and mutually exclusive protected execution. When observing performance improvements through threading without locks, runtime decreased steadily as more threads were added. As expected in our test cases, utilizing more threads resulted in shorter runtimes. However, the comparison between hist1.txt and hist2.txt revealed massive differences. multiple threads read and overwrote the same shared memory addresses concurrently, which result in thousands of redundant Collatz calculations and a corrupted histogram.

Additionally, the impact of using locks restricted performance. While our diff test between hist1.txt (single-threaded) and hist3.txt (multi-threaded with locks) showed absolutely zero differences, the runtime cost was massive. The locked implementation achieved its best runtime at $T = 3$ (1.51s, a $1.66\times$ speedup). Beyond 3 threads, performance decreased.

Conclusions

Parallel computation reduces execution time by using the multi-core CPU architecture. In the Xeon environment, independent cores processed concurrent loops simultaneously, allowing the hardware to natively distribute the computational load. However, because our threads operated within a shared memory architecture, parallel execution without synchronization led to race conditions.